

GRAF GECISLERI

Graf Turleri · DFS (Rekursif + Iteratif) · BFS

Temel: $G = (V, E)$, $E \subseteq V \times V$, $|E| \leq |V|^2$

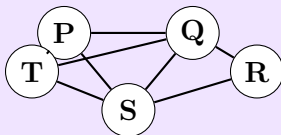
Ornek: $V = \{P, Q, R, S, T\}$ $\deg(P) = 3$, $\deg(Q) = 4$, $\deg(R) = 2$, $\deg(S) = 4$, $\deg(T) = 3$

1. Graf Turleri

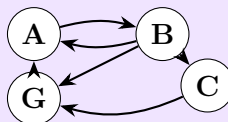
Graf Temel Kavramlari ve Turleri

Graf Turu	Ozellik
Onemsiz (Trivial)	Tek dugum, hic kenar yok
Bos (Null)	Dugumler var ama hic kenar yok
Basit (Simple) / Rast	Oz dongu (self-loop) yok, coklu kenar yok
Yonsuz (Undirected)	Baglantilarin yonu yoktur
Yonlu (Directed)	Baglantilarin yonu vardır
Coklu (Multi)	iki node arasinda birden fazla baglanti olabilir
Sozde (Pseudo)	Oz dongu vardır
Bagli (Connected)	Her dugumden diger dugume ulasabilirsiniz
Baglantirsiz	Bagimsiz bileşkenler vardır
Tam (Complete)	Her kose cifti arasinda 1 kenar vardır
Duzenli (Regular)	Tum dugumlerin derecesi esittir ($\deg = 2$, $\deg = 3$, ...)
Dogusel (Planar)	Kenarlar birbirini kesmez
Dongusel (Cyclic)	Icerisinde cevre (dongu) vardır
Cevrimsiz (Acyclic)	Dongu icermeyen
Sonlu	Sonlu sayida kose ve kenar
Sonsuz	Sonsuz sayida kose ve kenar

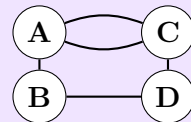
Yonsuz



Yonlu



Coklu



Dongü ve Dairesel Graflar: En az 3 kose, her kosenin derecesi 2, bir zincir icinde bagli. (Circuits or Cycle)

Hamilton Grafi: Her koseyi tam olarak 1 kez ziyaret eden ve baslangic kosede biten kapali bir yol (Hamilton dongusu) iceren graftir. *NP-Complete – polinom zamanda cozulemez.*

Euler Grafi: Sonlu bir grafta her kenari tam olarak 1 kez gecen ve koselerin yeniden ziyaret edilmesine izin veren bir yoldur – Euler yolu. Ayni kosede baslayip biten bir Euler yoluna **Euler dongusu** denir. Graf Euler yoluna sahip ama Euler dongusu yoksa **yari Euler grafi** denir. Bunlar ilk kez 1736'da Leonhard Euler tarafından Konigsberg yedi koprusu problemini cozerken tartisilmistir.

Ornek Euler yolu: $D \rightarrow E \rightarrow B \rightarrow C \rightarrow A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F \rightarrow D$

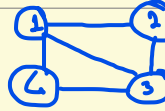
Kural: Euler dongusu icin tum dugumlerin derecesi cift olmalidir.

Pratik – Graf Turleri

(a) $V = \{1, 2, 3, 4\}$, $E = \{(1, 2), (2, 3), (3, 4), (4, 1), (1, 3)\}$ icin: Derece dizisini yazin, tam graf mi, Euler dongusu var mi? _____

hayır

var



(b) Hamilton ile Euler grafinin temel farki nedir? _____

köşe

kenar

(c) Asagidaki grafin Euler dongusu var mi? Aciklayin. $V = \{A, B, C, D\}$, $E = \{(A, B), (B, C), (C, D), (D, A), (A, C)\}$ _____



var

2. DFS – Derinlige Oncelikli Arama

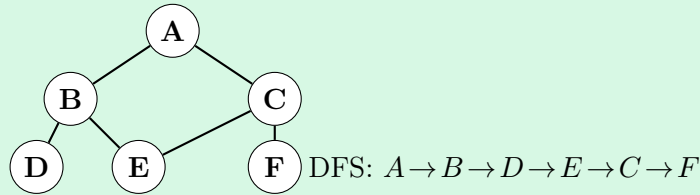
DFS her zaman bulunduđu dugumden mumkun oldugca derine gitmeye calisir. Standart bir DFS uygulaması grafin her kosesini iki kategoriden birine koyar: **ziyaret edilen**, **ziyaret edilmeyen**.

DFS – Derinlige Oncelikli Arama

Iteratif DFS Algoritma Adimlari:

1. Grafin koselerinden herhangi birini bir yigin (stack) ustune koyarak baslayin.
2. Yigin en ust ogesini alin ve ziyaret edilenler listesine ekleyin.
3. Bu kosenin komsularinin bir listesini dusunun; ziyaret edilenler listesinde olmayanlar yiginin en ustune ekleyin.
4. Yigin bosalana kadar 2. ve 3. adimlari tekrarlayin.

Ornek Graf (A-B-C-D-E-F):



Rekursif DFS:

```

static void recursiveDFS(int node) {
    visited[node] = true;
    System.out.println("Visited: " + node + " (recursive)");
    for (int i = 0; i < numNeighbors[node]; i++) {
        int neighbor = graph[node][i];
        if (!visited[neighbor])
            recursiveDFS(neighbor);
    }
}

```

Iteratif DFS:

```

static void iterativeDFS(int start) {
    Stack<Integer> stack = new Stack<>();
    stack.push(start);
    while (!stack.isEmpty()) {
        int current = stack.pop();
        if (!visited[current]) {
            visited[current] = true;
            System.out.println("Visited: " + current + " (iterative)");
            for (int i = numNeighbors[current] - 1; i >= 0; i--) {
                int neighbor = graph[current][i];
                if (!visited[neighbor])
                    stack.push(neighbor);
            }
        }
    }
}

```

addEdge ve resetVisited yardımcı metodları:

```

static void addEdge(int u, int v) {
    graph[v][numNeighbors[v]++] = u;
    // graph[u][numNeighbors[u]++] = v; // yonsuz icin bu satiri da ac
}

static void resetVisited(int numNodes) {
    for (int i = 0; i < numNodes; i++)
        visited[i] = false;
}

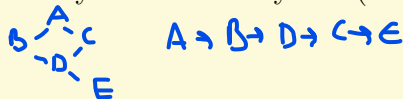
```

Karmasiklik:

- Bellek karmasikligi: $O(V)$ (her dugum bir kez ziyaret edilir)
- Zaman karmasikligi: $O(V + E)$ (tum dugum ve kenarlar islenir)

Pratik – DFS

(a) $V = \{A, B, C, D, E\}$, $E = \{(A, B), (A, C), (B, D), (C, D), (D, E)\}$ grafinde A'dan başlayan DFS izini yazın (alfabetik sirada komsu sec). _____



(b) Rekursif DFS ile iteratif DFS her zaman ayni sirayi verir mi? Aciklayin. _____

3. BFS – Genislige Oncelikli Arama

BFS her zaman bulunduđu konumdan seviye seviye genise dogru ilerler. Kuyruk (queue) kullanir. Agirsiz graflar icin en kisa yolu garanti eder.

BFS – Genislige Oncelikli Arama

BFS Algoritma Adimlari:

1. Baslangic dugumunu kuyrugun sonuna ekle ve visited olarak isaretle.
2. Kuyruk bos degilken:
 - a) Kuyruktan bir dugum cikar.
 - b) Dugunun komsusu: erzana yaz veya listeye ekle.
 - c) Tum ziyaret edilmemis komsuyu kuyrugun sonuna ekle ve isaretle.
3. Tum dugumler islenene kadar devam et.

BFS Kodu:

```
static void BFS(int start, int numNodes) {
    boolean[] visited = new boolean[numNodes];
    Queue<Integer> queue = new LinkedList<>();
    visited[start] = true;
    queue.add(start);
    while (!queue.isEmpty()) {
        int current = queue.poll();
        System.out.print(current + " ");
        for (int neighbor : adj[current]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue.add(neighbor);
            }
        }
    }
}
```

DFS vs BFS:

	Yapi	Zaman	Bellek	En kisa yol?
DFS	Yigin (Stack)	$O(V + E)$	$O(V)$	Hayir
BFS	Kuyruk (Queue)	$O(V + E)$	$O(V)$	Evet (agirsiz)

BFS Avantajlari:

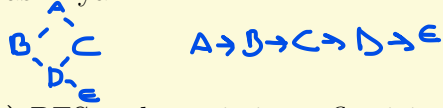
- Nispeten dusuk zaman karmasikligina sahiptir.
- Yonlu ve yonsuz graflar icin calisir.

BFS Dezavantaji:

- $|E|$ 'nin $|V|$ 'ye yakin oldugu yogun graflar icin her bir dugum cifti icin en kisa yolu hesaplamak icin uygun degildir.

Pratik – BFS

(a) Ayni graf üzerinde (A 'dan baslayan BFS) her dugumun kesfedildigi seviyeyi ve BFS sirasini yazin. _____



(b) BFS neden agirsiz graflar icin en kısa yolu garanti eder? 1'er kenar uzaktayarak
taradigi icin.

(c) Asagidaki durumda DFS mi BFS mi tercih edersiniz? Neden? “Bir labirentin cikisini bulmak istiyoruz.” BFS → daha kısa yolu bulur.

Ozet:

- $G = (V, E)$, $|E| \leq |V|^2$ – Graf ne kadar cok kenar icerirse o kadar yogun.
- DFS: Yigin kullanir, derine gider, dongu tespiti icin idealdir.
- BFS: Kuyruk kullanir, genise gider, en kısa yol (agirsiz) icin idealdir.
- Her iki algoritmanin da zaman karmasikligi $O(V + E)$, bellek $O(V)$.
- Hamilton: Her koseyi 1 kez gec (NP-Complete). Euler: Her kenari 1 kez gec (verimli cozulebilir).